

[81] Practical Selectivity Estimation Through Adaptive Sampling

요약

Lipton, Naughton, Schneider의 SIGMOD 1990 논문으로, 실무에서 적용 가능한 적응적 샘플링(adaptive sampling) 방법을 제시한 '이정표' 논문이다. 이전 논문들([80])이 샘플링의 이론을 정립했다면, 본 논문은 현실의 불확실성(미지의 선택도, 시간 제약, 메모리 제약) 속에서 어떻게 실질적으로 동작하는 샘플링을 구현할지 보여준다.

핵심 기여: 1. **적응적 샘플링 알고리즘**: 샘플 크기를 동적으로 조정하여 필요한 만큼만 샘플링 2. **점진적 신뢰도 검증**: 샘플링 과정 중 신뢰도를 평가하여 언제 멈출지 결정 3. **실용성**: 데이터베이스 옵티마이저에 직접 통합 가능한 형태

논문의 핵심은 "적응적"이라는 점이다. 사전에 σ 를 모를 때, 초기 샘플로 σ 를 대략 추정한 후, 이 추정값에 따라 필요한 추가 샘플 크기를 결정한다. 이를 통해 샘플링 비용을 크게 절감할 수 있다.

본 논문과의 관계: Exqutor 논문이 직접 "적응적 샘플링"을 수용하는 방식의 이론적 원천. Exqutor의 핵심 메커니즘인 "범위 필터 조건과 벡터 검색 조건의 결합 선택도를 적응적으로 샘플링으로 추정"은 이 논문의 방법론을 직접 구현한 것이다.

상세분석

81.1 문제의 현실성

샘플 크기 결정의 딜레마:

공식 (논문 [80]): $n = z^2 \times \sigma(1-\sigma) / \epsilon^2$

문제: σ 를 모른다!

선택지 1: 최악 경우 가정 ($\sigma = 0.5$)

- $n_{\max} = z^2 \times 0.25 / \epsilon^2$
- 매우 큰 샘플 크기 (보수적)
- 시간/비용 낭비

선택지 2: 추측 ($\sigma \approx 0.1?$)

- $n = z^2 \times 0.09 / \epsilon^2$
- 작은 샘플
- 신뢰도 부족 위험

해결: 적응적 샘플링

- 초기 샘플로 σ 추정
- 이 추정값에 따라 추가 샘플링 여부 결정

81.2 적응적 샘플링 알고리즘

기본 알고리즘:

```

def adaptive_sampling_estimate(query, target_precision=0.15,
                              confidence_level=0.90):
    """
    적응적 샘플링으로 선택도 추정

    목표:  $\sigma$ 를  $\pm \text{target\_precision} \times \sigma$  범위에서 90% 신뢰도로 추정
    """

    # 1단계: 초기 샘플
    sample_size = 100 # 초기값
    sample = data.sample(n=sample_size)
    results = query.execute(sample)
    match_count = count_matches(results)

    # 2단계: 초기 추정
    sigma_hat = match_count / sample_size

    # 3단계: 신뢰도 검증
    while True:
        # 현재 샘플로 달성 가능한 신뢰도 계산
        se = math.sqrt(sigma_hat * (1 - sigma_hat) / sample_size)
        margin_of_error = z_score * se # z_score = 1.645 (90%)
        relative_error = margin_of_error / sigma_hat

        # 4단계: 목표 정확도 달성 여부 확인
        if relative_error <= target_precision:
            break # 충분한 신뢰도 달성

        # 5단계: 필요한 총 샘플 크기 계산
        required_sample = int((z_score / target_precision) ** 2 *
                               sigma_hat * (1 - sigma_hat))
        additional_sample = required_sample - sample_size

        # 6단계: 추가 샘플링
        if additional_sample <= 0:
            break # 최적화: 이미 충분함

        new_sample = data.sample(n=additional_sample)
        new_results = query.execute(new_sample)
        match_count += count_matches(new_results)
        sample_size += additional_sample

        #  $\sigma$  재추정
        sigma_hat = match_count / sample_size

        # 7단계: 안전 장치 (무한 루프 방지)
        if sample_size > len(data) * 0.5: # 50% 이상 샘플링
            break

```

```
return sigma_hat, sample_size
```

알고리즘의 유연성:

초기 샘플에서 σ_{hat} 추정:

$\sigma_{\text{hat}} = 0.5$ (50%):

- 매우 큰 샘플 필요 (최악의 경우)
- 추가 샘플링 수행

$\sigma_{\text{hat}} = 0.1$ (10%):

- 중간 크기 샘플 필요
- 일부 추가 샘플링 가능

$\sigma_{\text{hat}} = 0.01$ (1%):

- 큰 샘플 필요? 하지만 이미 100개 중 1개면...
- 신뢰도 저하 → 큰 샘플링 필요

$\sigma_{\text{hat}} = 0.001$ (0.1%):

- 극도로 큰 샘플 필요
- 또는 샘플링 포기, 다른 방법 사용

81.3 동적 정밀도 조정

점진적 신뢰도 평가:

반복 i 에서:

샘플 크기: n_i

추정된 σ : σ_{hat_i}

표준오차: $SE_i = \sqrt{(\sigma_{\text{hat}_i} \times (1 - \sigma_{\text{hat}_i}) / n_i)}$

상대오차: $RE_i = 1.645 \times SE_i / \sigma_{\text{hat}_i}$ (90% 신뢰도)

반복별 진행:

반복 1: $n=100$, $\sigma_{\text{hat}}=0.08$, $SE=0.027$, $RE=44\%$ (매우 높음)
→ 추가 샘플링 필요

반복 2: $n=500$, $\sigma_{\text{hat}}=0.12$, $SE=0.012$, $RE=14\%$ (목표 15% 달성!)
→ 중단 가능

81.4 특수한 경우들

매우 낮은 선택도 처리:

$\sigma_{\text{hat}} = 0.001$ (0.1%)인 경우:

$$\begin{aligned}\text{공식: } n &= 1.645^2 \times 0.001 \times 0.999 / (0.15 \times 0.001)^2 \\ n &= 2.706 \times 0.000999 / 0.000000225 \\ n &\approx 120,000\end{aligned}$$

원래 데이터가 1,000,000행이면:

$$\text{샘플율} = 120,000 / 1,000,000 = 12\%$$

대체 전략:

1. 정밀도 요구사항 완화 (RE = 50% 허용)
→ 필요 샘플: 4,800 (0.5%)
2. 신뢰도 낮춤 (80% 신뢰도로)
→ 필요 샘플: 68,000 (6.8%)
3. 샘플링 포기 → 통계 기반 추정 전환

제로 매칭(Zero Matches):

샘플 100개 중 0개가 조건 만족:

$$\sigma_{\text{hat}} = 0$$

문제: $SE = \sqrt{(0 \times 1 / 100)} = 0$ (undefined)

해결책:

1. 라플라스 평활(Laplace Smoothing): $\sigma_{\text{hat}} = 1 / (n+2)$
→ 100개 샘플 시 $\sigma_{\text{hat}} = 1/102 \approx 0.98\%$
2. 추가 샘플링 강제 실행
→ 더 큰 샘플에서 일치 찾을 때까지

81.5 멀티쿼리 최적화

여러 쿼리를 동시에 추정할 때:

쿼리 Q1: WHERE condition1

쿼리 Q2: WHERE condition2

방법 1: 독립적 샘플링

각 쿼리에 대해 별도로 샘플링

총 샘플 비용: 큼 (각각 독립적)

방법 2: 공유 샘플 (Shared Sampling)

하나의 샘플 S에서 모든 쿼리 실행

샘플 S 선택:

$Q1_matches = |\{r \in S : condition1(r)\}|$

$Q2_matches = |\{r \in S : condition2(r)\}|$

$\sigma_hat_1 = Q1_matches / |S|$

$\sigma_hat_2 = Q2_matches / |S|$

장점: 샘플 한 번만 수집

단점: 개별 쿼리 정밀도 조정 어려움

81.6 비용 모델과의 통합

샘플링 비용 vs 추정 정확도:

총 비용 = 샘플링 비용 + 부정확한 추정으로 인한 손해

샘플링 비용:

$C_sample = \text{샘플 크기} \times \text{행당_처리_시간}$

부정확 비용:

$C_error = |\sigma_hat - \sigma| \times \text{최악_실행_계획_비용}$

최적 샘플 크기: $C_sample + C_error$ 최소화

예:

정밀도 $\pm 50\%$ 오류 허용:

→ 샘플 100개 (빠름)

→ 부정확하면 최악 계획 선택 가능성

정밀도 $\pm 10\%$ 오류 요구:

→ 샘플 2,500개 (느림)

→ 정확한 계획 선택으로 큰 이득

81.7 본 논문과의 관계

Exqutor의 적응적 샘플링의 원천:

Exqutor 논문의 "적응적 샘플링" 방식이 정확히 이 논문([81])의 알고리즘을 구현한 것이다:

Exqutor 아키텍처:

범위 필터 + 벡터 검색의 결합 선택도 추정

쿼리: WHERE price $\in$ [100, 500] AND similarity(embedding) > 0.8

1단계: 초기 샘플 (전체의 1%, 약 10,000행)

```
S = data.sample(frac=0.01)
```

2단계: 샘플에서 두 조건 모두 만족하는 행 수 세기

```
matches = count(S, price_condition AND vector_condition)
```

3단계: 초기 선택도 추정

```
 $\sigma_{\text{hat}} = \text{matches} / \text{len}(S)$ 
```

4단계: 신뢰도 계산 (논문 [80]의 공식)

```
se =  $\sqrt{(\sigma_{\text{hat}} \times (1 - \sigma_{\text{hat}})) / \text{len}(S)}$ 
```

```
confidence = 1.645  $\times$  se /  $\sigma_{\text{hat}}$  (90% 신뢰도)
```

5단계: 목표 정밀도(예: $\pm 15\%$) 달성 여부 확인

```
if confidence <= 0.15:
```

```
    → 충분한 샘플 확보, 사용 가능
```

```
else:
```

```
    → 추가 샘플링 필요
```

6단계: 추가 샘플링 (필요한 경우만)

```
required_total =  $(1.645 / 0.15)^2 \times \sigma_{\text{hat}} \times (1 - \sigma_{\text{hat}})$ 
```

```
additional = required_total - len(S)
```

```
if additional > 0:
```

```
    S_new = data.sample(frac=additional/N)
```

```
    matches_new = count(S_new, both_conditions)
```

```
 $\sigma_{\text{hat\_final}} = (\text{matches} + \text{matches\_new}) / (\text{len}(S) + \text{len}(S_{\text{new}}))$ 
```

7단계: 최종 선택도 사용

옵티마이저가 $\sigma_{\text{hat_final}}$ 을 기반으로 실행 계획 선택

Exqutor의 창의성:

기준 (논문 [81]):

- 단일 쿼리의 선택도 추정

Exqutor의 확장:

- 두 이질적 조건(범위 필터 + 벡터 검색)의 결합 선택도
- $\sigma_{total} = \sigma_{range} \times \sigma_{vector}$ (독립성 가정)
- 적응적 샘플링으로 양쪽 조건을 동시에 평가

81.8 현대적 적용 사례

Exqutor의 다층 구조:

L1: 경량 모델 (논문 [70])

범위 필터의 선택도를 빠르게 추정

비용: <1ms

정확도: $\pm 20\%$

L2: 적응적 샘플링 (논문 [81])

L1의 추정이 불확실할 때 사용

비용: 10-100ms (샘플 크기에 따라)

정확도: $\pm 10\%$

L3: 전체 스캔 (정확한 계산)

선택도가 매우 중요한 경우만

비용: 1-10초 (전체 데이터 스캔)

정확도: 100%

선택 로직:

```
if L1_confidence > 0.95:
```

```
    사용 L1 (빠름)
```

```
elif time_budget > 100ms:
```

```
    사용 L2 (적응적 샘플링)
```

```
else:
```

```
    사용 L1 (신뢰도 낮음)
```

추가 제기 문제

1. 초기 샘플 크기: 초기값 100개가 항상 적절한가? 데이터 특성에 따라 조정할 수 있는가?
2. 반복 중단 조건: 신뢰도 기준 외에 다른 중단 조건이 필요한가? (시간, 메모리, 비용 등)
3. 샘플 편향: 데이터가 비균등하게 분포되어 있을 때(예: 클러스터링), 무작위 샘플링이 정확한가?

4. **벡터 유사도의 선택도**: 벡터 검색에서 "top-k"는 선택도가 정의되기 어려운데(k는 고정, 유사도 임계값은 데이터 의존적), 어떻게 샘플링할 것인가?
5. **상관성이 있는 경우**: 범위 필터 속성과 벡터 임베딩이 강하게 상관되어 있을 때, $\sigma_{total} = \sigma_{range} \times \sigma_{vector}$ 가정이 성립하는가?
6. **실시간 적응성**: 데이터가 시간에 따라 변할 때, 과거 샘플이 유효한가? 주기적 재샘플링이 필요한가?
7. **멀티 필터 확장**: (range_filter1 AND range_filter2 AND vector_search)에서 3중 곱셈 $\sigma_{total} = \sigma_1 \times \sigma_2 \times \sigma_3$ 가 성립하는가?

논문 [81]의 역사적 의의

이 논문은 1990년에 발표되었지만, 30년이 지난 지금도:

1. **학부 데이터베이스 강의**에서 샘플링 기반 카디널리티 추정의 기초로 가르쳐짐
2. **PostgreSQL, SQL Server 등 상용 DBMS**에서 기본 전략으로 채택
3. **현대 벡터 DB와 하이브리드 검색 시스템**의 선택도 추정 기법으로 여전히 핵심

Exqutor가 "적응적 샘플링"을 사용하기로 한 것은, 단순한 구현 선택이 아니라, 30년의 검증을 거친 이론적으로 견고한 방법론을 채택한 것이다. 이는 Exqutor의 설계 철학의 깊이를 보여준다.